

The Clarke & Wright savings heuristic for the capacitated VRP

Sequential and parallel versions, tested on the 33 benchmark instances of Juan et al. (2011).

Jesús Inurria Porrero · Javier Salvatierra Corchado

Heuristic Algorithms in Transport and Finance · Master's course · September 2026

One depot, many vehicles, limited capacity

The VRP generalises the TSP: a fleet leaves one depot, every customer is served by **exactly one** vehicle, and every vehicle comes back. In the **capacitated** version each vehicle carries at most Q and customer i consumes q_i .

```
min   $\Sigma$  length of every route
s.t.  every customer on exactly one route
       $\Sigma q_i \leq Q$  on every route
      every route starts and ends at 0
```

It contains the TSP (one vehicle, $Q = \infty$), so it is NP-hard: heuristics that give near-optimal routes quickly are the practical tool.

NP-hard

no polynomial algorithm is known

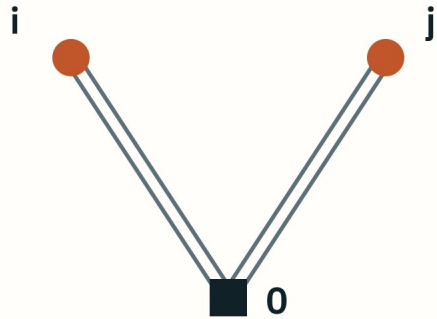
33

benchmark instances selected by Juan et al. (2011), six families A B E F M P

22 → 135

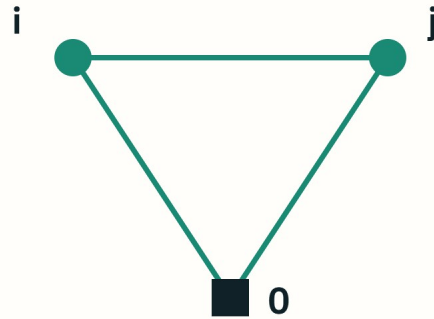
nodes; A-n32-k5 = 32 nodes, best known with $k = 5$ vehicles; Q from Table 1 of the paper

Formulation: what a merge saves



Two trips

$$2c(0,i) + 2c(0,j)$$



One trip

$$c(0,i) + c(i,j) + c(0,j)$$

Depot 0, demands $q_i \leq Q$, Euclidean distances $c(i,j)$, fleet size not fixed.

Serving i and j on one route replaces the return $i \rightarrow 0$ and the departure $0 \rightarrow j$ by the edge $i \rightarrow j$.

Triangle inequality: $c(i,j) \leq c(i,0) + c(0,j)$, so $s(i,j) \geq 0$. It is largest for customers close to each other and far from the depot.

SAVING (CLARKE
& WRIGHT, 1964)

$$s(i,j) = c(0,i) + c(0,j) - c(i,j)$$

Clarke & Wright: the dummy solution

Start from the worst feasible solution: **one vehicle per customer**, $0 \rightarrow i \rightarrow 0$. Every customer is visited once and receives q_i , and it is feasible because every $q_i \leq Q$.

- 1 $s(i,j)$ for every pair of customers
- 2 sort by decreasing saving $\rightarrow$ savings list
- 3 one route $0 \rightarrow i \rightarrow 0$ per customer
- 4 merge routes along the list (next)

31

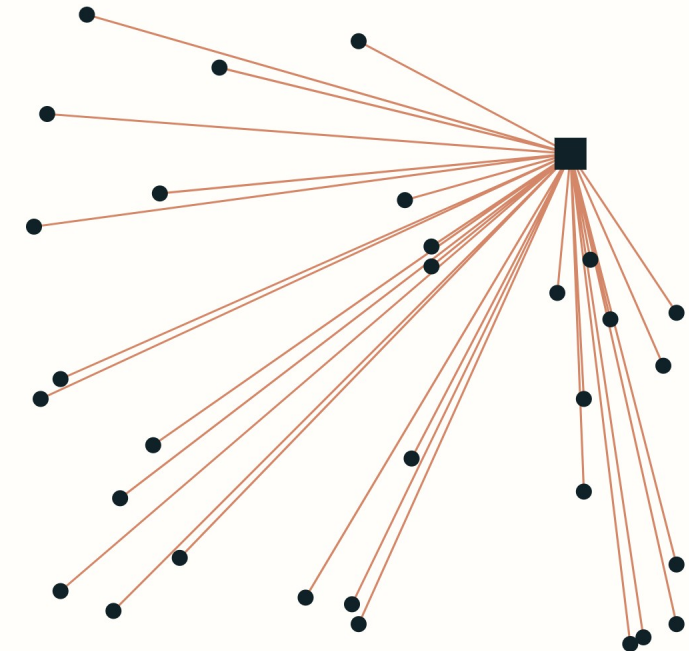
routes, one per customer

3 740.76

cost = $2 \sum c(0,i)$

465

edges in the savings list



A-n32-k5: 31 customers, capacity $Q = 100$;
square = depot

Clarke & Wright: improving the solution

Walk the savings list from the largest saving down. For an edge (i, j) , merge the route of i and the route of j through $i \rightarrow j$ if **all three** conditions hold:

1 • Different routes

If i and j already share a route, the new edge closes a loop that never visits the depot.

2 • Both exterior

i and j are next to the depot, at an end of their routes. An interior customer already has two neighbours.

3 • Capacity

The joint load of the two routes does not exceed Q .

A merge replaces $(i, 0)$ and $(0, j)$ by (i, j) : the cost drops by **exactly** $s(i, j)$.

Always taking the best remaining edge makes the heuristic greedy.

A rejected edge is dropped for good: routes only grow, so its conditions can only get worse.

A-N32-K5 • FIRST EDGE (4, 11)

saving 190.45

cost 3 740.76 → 3 550.31

route [4, 11], load 33

Implementation: three common objects

routes

```
{1: [1], 2: [2], ...}
```

Route index $\rightarrow$ list of customers. The depot at both ends is implicit.

route_of

```
route_of[i]  $\rightarrow$  route of i
```

Array over the nodes: the route of a customer is found in $O(1)$.

load

```
load[r] =  $\sum q_i$  on r
```

Demand carried by each route: the capacity check is $O(1)$.

SHARED BUILDING BLOCKS · VRP.PY

make_savings_list	stable sort: ties keep (i, j)
construct_initial_sol	the dummy solution
check_merging_conditions	the three conditions
merge_routes_using_edge	reverse, join, relabel, add loads

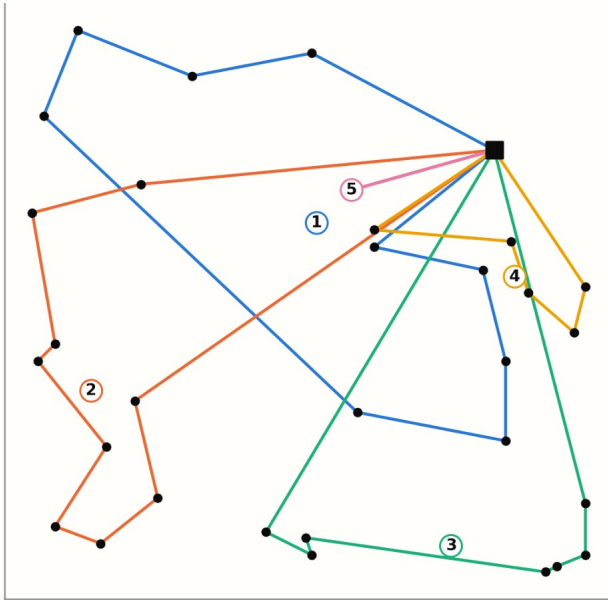
SCOPE

Classic, deterministic CWS: lines 1–2 of Figure 4 in Juan et al. (2011).

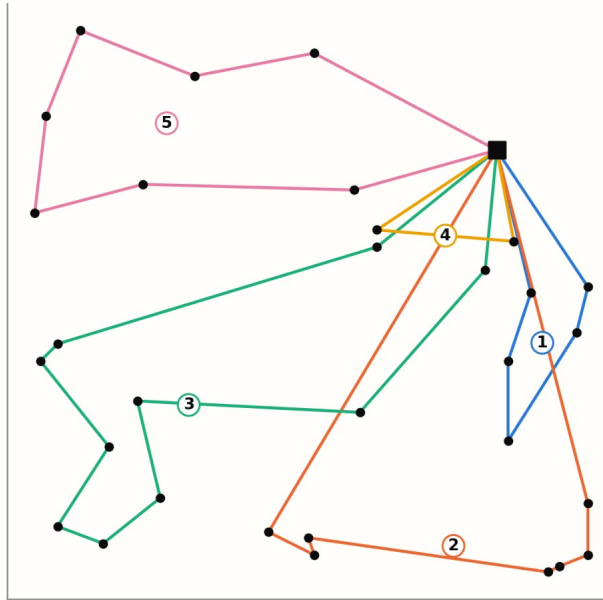
No random edge choice, no route cache, no splitting.
Python 3.14 + NumPy.

Sequential vs. parallel on A-n32-k5

Sequential CWS — 907.11



Parallel CWS — 843.69



SEQUENTIAL

907.11

+7.52 % vs. JORS. One route at a time: the last routes collect the leftovers; route 5 serves one customer.

PARALLEL

843.69

Equal to the JORS CWS. Any two routes can merge, so all routes grow together.

NOTE

The parallel CWS is also implemented in C++, function by function: the same 33 solutions, in 13 μ s – 1.38 ms per instance.

Results: sequential vs. parallel

9.97 %

mean cost reduction of the parallel CWS over the sequential one

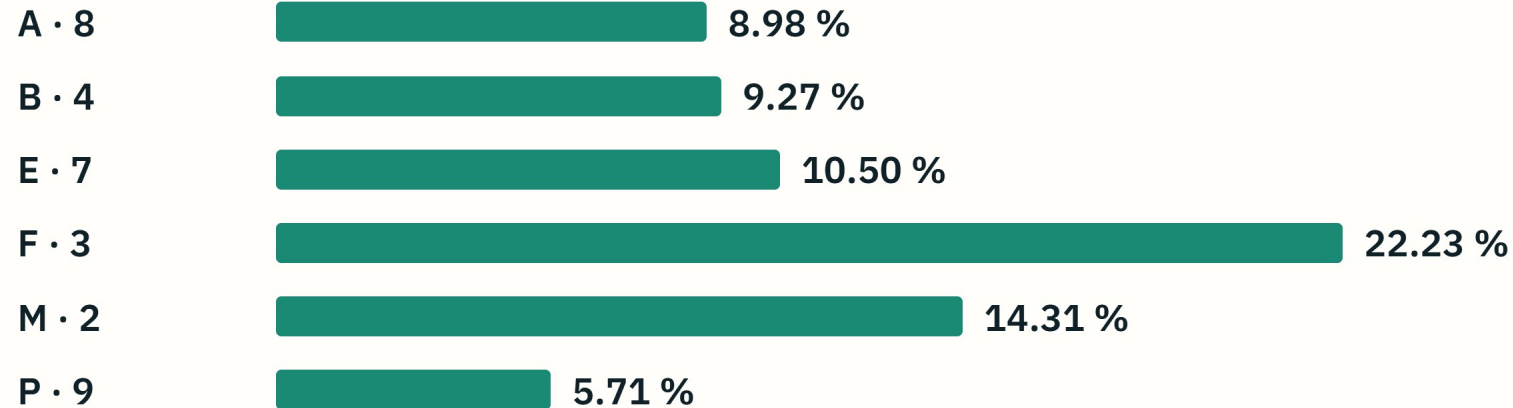
32 / 33

instances where the parallel CWS is cheaper; the exception is P-n40-k5

3.4× faster

in Python: 4.4 ms vs. 15.1 ms mean per instance

Cost reduction of the parallel CWS over the sequential one, mean per family (higher = parallel better)



Results: parallel vs. JORS

27 / 33

instances with exactly the CWS cost of Juan et al. (2011), Table 1

33 / 33

if tied savings are taken in reverse order: the six differences come only from ties

0.04 %

mean gap to JORS; 4.91 % to the best-known solutions (JORS CWS: 4.87 %)

The six instances that differ

Instance	Ours	JORS	Gap
E-n76-k7	738.13	737.74	+0.05 %
E-n76-k10	907.39	900.26	+0.79 %
E-n76-k14	1 054.60	1 073.43	-1.75 %
P-n50-k10	739.84	734.32	+0.75 %
P-n76-k4	688.34	689.13	-0.11 %
P-n76-k5	709.38	698.51	+1.56 %

Negative gap = our solution is cheaper. Our list keeps tied savings in generation order (i, j); neither order is better in general.

CONCLUSION

**One greedy pass over the savings list:
always feasible, no parameters, about 5 %
above the best known, in milliseconds.**

WORTH REMEMBERING

Parallel beats sequential by about 10 %,
most on the clustered F instances.

Tied savings decide 6 of the 33
instances: the order of the list is
not neutral.

Next step: randomise the edge choice
and repeat, as Juan et al. (2011) do.

Thank you — questions?